

Especificação do Sistema

Levantamento de requisitos, regras de negócio, modelagem de dados, arquitetura e descrição dos módulos da plataforma de reserva de arenas esportivas.

Stack: Laravel 12 · PHP 8.3 · MySQL/InnoDB · Bootstrap 5

Papéis: Cliente · Proprietário · Funcionário · Admin Tabelas de domínio: 20

5 Levantamento de Requisitos

5.1 Requisitos Funcionais

CÓDIGO	REQUISITO	DESCRIÇÃO
RF-A · ACESSO E CONTA		
RF-A1	Navegação pública	Visitante navega e pesquisa arenas ativas e vê os detalhes (galeria, quadras, horários, formas de pagamento e política de cancelamento) sem precisar de login.
RF-A2	Cadastro de cliente	Criar conta com nome, e-mail único, senha, telefone e aceite dos termos.
RF-A3	Cadastro de proprietário	Assistente de 4 etapas: você/empresa, arena, funcionamento e quadras — cria conta, empresa e a 1ª arena.
RF-A4	Cliente vira proprietário	Usar a conta atual (encerra o papel de cliente) ou criar dados de acesso novos (mantém o cliente). O histórico é preservado.
RF-A5	Login e logout	Autenticação com limite de tentativas.
RF-A6	Recuperação de senha	Link enviado ao e-mail com tela de redefinição.
RF-A7	Verificação de e-mail	Confirmação por link no cadastro — implementada e ativável por configuração.
RF-A8	Perfil e conta	Cada papel edita os próprios dados pessoais e a senha; o cliente pode excluir a conta.
RF-B · CLIENTE – RESERVAS		
RF-B1	Listar e favoritar arenas	Pesquisar arenas e favoritar/desfavoritar sem recarregar a página.
RF-B2	Galeria da arena	Ver as fotos da arena com ampliação em tela cheia.
RF-B3	Reservar quadra	Escolher a quadra, a data e um horário disponível.
RF-B4	Pagamento online	Pagar a reserva por PIX ou cartão (simulado).
RF-B5	Cancelar reserva	Grátis, com taxa (paga online) ou paga com reembolso (pago – taxa), conforme a política da arena.

CÓDIGO	REQUISITO	DESCRIÇÃO
RF-B6	Reagendar / editar	Alterar a reserva enquanto a política permitir.
RF-B7	Minhas reservas	Consultar por situação: a realizar hoje pendentes histórico canceladas não pagas .
RF-B8	Notificações e sugestões	Ler notificações do sino; enviar sugestão ou reportar um bug.
RF-C · PROPRIETÁRIO – ARENA, QUADRAS E EQUIPE		
RF-C1	Gerenciar arena	Cadastrar, editar (nome, contato, descrição) e excluir (soft delete: cancela reservas futuras com motivo e mantém o histórico).
RF-C2	Ativar / desativar arena	Ligar ou desligar a arena para novos agendamentos.
RF-C3	Horários de funcionamento	Definir até 2 períodos por dia.
RF-C4	Formas de pagamento	Selecionar as formas aceitas pela arena.
RF-C5	Taxa de cancelamento	Configurar fixa ou percentual; sempre ou por janela de horas antes do início.
RF-C6	Galeria de fotos	Adicionar (até 15), reordenar e definir a capa.
RF-C7	Gerenciar quadras	Criar, editar, ativar/desativar e excluir (soft delete), com os esportes de cada uma.
RF-C8	Múltiplas arenas	Escolher e trocar a arena ativa em gestão.
RF-C9	Gerenciar funcionários	Criar, editar, ativar/desativar e excluir gerentes e atendentes.
RF-C10	Gerenciar empresa	Desativar e reativar a própria empresa.
RF-D · CAIXA E FINANCEIRO		

CÓDIGO	REQUISITO	DESCRIÇÃO
RF-D1	Abrir / fechar caixa	Registrar saldo inicial e final.
RF-D2	Reserva presencial	Registrar reserva no balcão para cliente sem cadastro.
RF-D3	Operar reservas	Confirmar, cancelar e reagendar.
RF-D4	Receber com desconto	Receber o pagamento no caixa com desconto opcional (motivo obrigatório, gravado nos registros).
RF-D5	Lançamentos avulsos	Registrar entradas e saídas manuais.
RF-D6	Reembolsos e pendentes	Reembolsar cancelamento de reserva paga; lançar pagamentos/reembolsos retidos por o caixa estar fechado.
RF-D7	Relatórios	Balanço, lançamentos por mês e faturamento líquido real (reservas + cancelamentos + avulsas – despesas).
RF-D8	Listas de apoio	Reservas a receber, taxas a receber, pagamentos e reembolsos a lançar.
RF-E • CLIENTES E COMUNICAÇÃO (DONO/GERENTE)		
RF-E1	Lista de clientes	Clientes da arena com estatísticas (realizadas / pendentes / atrasadas) e filtro.
RF-E2	Detalhes do cliente	Reservas do cliente (a realizar, realizadas, não pagas, canceladas), com o abatimento dos descontos.
RF-E3	Mensagens	Enviar mensagem a um cliente ou em massa.
RF-F • FUNCIONÁRIOS		
RF-F1	Painel do gerente	Reaproveita as telas do dono para gerir a arena onde trabalha.
RF-F2	Painel do atendente	Consulta arena/quadras (por modal) e opera reservas e caixa — sem gestão, lucro ou funcionários (middleware <code>pode.gerir</code>).

CÓDIGO	REQUISITO	DESCRIÇÃO
RF-F3	Notificações	Os avisos do sino também chegam aos funcionários.
RF-G · ADMINISTRADOR		
RF-G1	Painel geral	Indicadores gerais da plataforma.
RF-G2	Gerenciar usuários	Ver, bloquear/desbloquear e excluir.
RF-G3	Gerenciar empresas	Proprietários/empresas: ver, ativar/desativar e excluir.
RF-G4	Gerenciar arenas	Ativar/desativar e excluir arenas; ativar/desativar/excluir quadras.
RF-G5	Ver clientes	Clientes por arena e por empresa.
RF-G6	Moderar feedbacks	Mudar o status de sugestões e bugs.
RF-G7	Aparência da tela inicial	Slides do carrossel: adicionar, reordenar, ativar e excluir.
RF-G8	Admins e pesquisa	Gerenciar administradores e usar a pesquisa rápida.
RF-H · REGRAS DE NEGÓCIO TRANSVERSAIS		
RF-H1	Reserva sem cadastro	Reserva presencial pode não ter cliente cadastrado; o nome vem do cadastro ou do balcão (<code>guest_name</code>).
RF-H2	Numeração amigável	Sequência por cliente e por arena, sem expor o id global do banco.
RF-H3	Sem sobreposição	Índice único (coluna gerada <code>slot_ativo</code>) impede reserva no mesmo horário/quadra.
RF-H4	Histórico preservado	Soft delete mantém nos registros itens excluídos (usuário, cliente, arena, quadra), com o nome.
RF-H5	Status automático	Confirmação e conclusão de reservas por prazo automático (agendador).

5.2 Requisitos Não Funcionais

CÓDIGO	REQUISITO	DESCRIÇÃO
RNF-S · SEGURANÇA		
RNF-S1	Senhas com hash	Armazenadas com bcrypt; nunca em texto puro.
RNF-S2	Recuperação segura	Token hashado no banco, expira em 60 min, uso único, throttle e resposta neutra (anti-enumeração de e-mail).
RNF-S3	Sessão protegida	2h por inatividade; trocar a senha derruba as outras sessões (AuthenticateSession global).
RNF-S4	Controle de acesso	Autorização por papel via middleware (<code>admin</code> , <code>pode.gerir</code> , <code>verified</code>); funcionário nasce verificado.
RNF-S5	Proteções da aplicação	5 tentativas de login/min, proteção CSRF e validação sempre no servidor.
RNF-S6	Upload seguro	Valida de verdade, re-codifica (remove EXIF/código escondido) e redimensiona a imagem.
RNF-U · USABILIDADE E ACESSIBILIDADE		
RNF-U1	Responsividade	Mobile-first (Bootstrap 5); tipografia fluida nas telas de gestão.
RNF-U2	Idioma	Interface 100% em português (pt_BR), inclusive mensagens de erro e validação.
RNF-U3	Feedback visual	Busca ao vivo, carrosséis, status coloridos e chips.
RNF-U4	Consistência de ações	Padrão de cores das ações (editar, detalhes, confirmar, cancelar).
RNF-P · DESEMPENHO E ESCALA		
RNF-P1	Paginação	Todas as listas paginam (tabelas e cards) — não carregam tudo de uma vez.
RNF-P2	Consultas eficientes	Eager loading evita N+1; índices nas colunas de filtro.

CÓDIGO	REQUISITO	DESCRIÇÃO
RNF-P3	Imagem otimizada	Compressão no cliente antes do upload.
RNF-P4	Escala mapeada	2 custos $O(n)$ anotados para dezenas de milhares (numeração via coluna, busca FULLTEXT).
RNF-C · INTEGRIDADE E CONFIABILIDADE		
RNF-C1	Transações	InnoDB; operações "tudo ou nada".
RNF-C2	Integridade referencial	31 chaves estrangeiras entre as tabelas.
RNF-C3	Preservação de dados	Soft delete mantém o histórico e evita quebra de FK.
RNF-C4	Sincronização	Agendador atualiza os status das reservas.
RNF-M · MANUTENIBILIDADE		
RNF-M1	Plataforma	Laravel 12 / PHP 8.3, arquitetura MVC.
RNF-M2	Versionamento do schema	Migrations versionam o banco; services e traits reutilizáveis.
RNF-M3	Documentação	Manual de instalação (HTML + PDF) e registro de decisões mantidos.
RNF-I · PORTABILIDADE, INSTALAÇÃO E PRODUÇÃO		
RNF-I1	Ambiente	Roda em WAMP/MySQL; <code>.env.example</code> testado.
RNF-I2	Configuração flexível	Cache/sessão em arquivo por padrão; tabelas prontas para banco em produção.
RNF-I3	Checklist de produção	SMTP, verificação de e-mail, fila (database + worker), <code>APP_DEBUG=false</code> e HTTPS — documentados.

6 Regras de Negócio

Restrições e políticas que o sistema garante, independentemente da interface.

RN01	Papel único por conta. Cada usuário é exatamente um tipo (<code>cliente</code> , <code>proprietário</code> , <code>funcionário</code> ou <code>administrador</code>). Um cliente pode virar proprietário reaproveitando a conta (encerra o papel de cliente) ou criando uma conta nova.
RN02	Horário exclusivo. Não pode haver duas reservas ativas na mesma quadra, data e horário (garantido por índice único na coluna gerada <code>slot_ativo</code>).
RN03	Origem da reserva. A reserva vem do site (cliente cadastrado e logado) ou é presencial (registrada no balcão, para cliente sem cadastro — dados de convidado).
RN04	Política de cancelamento. Cada arena define se cobra taxa, o tipo (fixa em R\$ ou percentual) e o modo (sempre, ou apenas dentro de uma janela de X horas antes do início).
RN05	Reembolso. Cancelar reserva já paga devolve valor pago – taxa ; cancelamento feito pela arena reembolsa integral. Reembolso em dinheiro é devolvido na arena; PIX/cartão estorna pela mesma forma.
RN06	Desconto no recebimento. Ao receber uma reserva no caixa pode-se aplicar desconto em R\$ com motivo obrigatório ; entra no caixa o valor líquido, e o desconto fica registrado.
RN07	Caixa aberto para lançar. Pagamentos e reembolsos só entram no caixa aberto. Com o caixa fechado ficam pendentes e são lançados manualmente na próxima abertura.
RN08	Faturamento real. O faturamento é o líquido do caixa (entradas – saídas: reservas + cancelamentos + avulsas – despesas), não a simples soma dos pagamentos.
RN09	Status automático. Reservas são confirmadas/concluídas por prazo automático (agendador); pendentes expiram se não confirmadas a tempo.
RN10	Exclusão preserva histórico. Arena, quadra, cliente e usuário usam <i>soft delete</i> : somem das listas ativas mas continuam nos registros (com o nome). Excluir arena/quadra cancela as reservas futuras com um motivo.
RN11	Alçada do funcionário. O gerente reaproveita as telas de gestão do dono; o atendente só opera reservas e caixa (sem gestão, lucro ou funcionários) — imposto pelo middleware <code>pode.gerir</code> .

RN12	Numeração amigável. As reservas têm numeração sequencial por cliente e por arena, sem expor o id global do banco.
RN13	Unicidade. E-mail único por usuário; CNPJ/CPF e razão social únicos por empresa; nome de quadra único por arena.
RN14	Pagamento válido. A forma de pagamento de uma reserva deve estar entre as aceitas pela arena; pagamento online só por PIX/cartão (dinheiro é na arena).
RN15	Edição limitada. O cliente só reagenda/edita a reserva enquanto o cancelamento ainda seria gratuito (antes do início / dentro da janela sem taxa).

7 Modelagem do Sistema

7.5 Modelo de Dados (DER)

20 tabelas de domínio e seus relacionamentos reais (chaves estrangeiras do banco). As 8 tabelas de infraestrutura do framework ficam fora do diagrama.


 um-para-muitos
 
 um-para-um (0..1)
PK primária
FK estrangeira
UK único

TABELA	CHAVES	DESCRIÇÃO
IDENTIDADE E PAPÉIS		
users	PK id · UK email	Conta única de acesso; <code>type</code> define o papel. Soft delete.
clients	FK user_id	Perfil de cliente (1:1 com user). Soft delete ao virar proprietário.
owners	FK user_id, deactivated_by	Proprietário/empresa (CNPJ/CPF e razão social únicos). Ativação e soft delete.
employees	FK user_id, arena_id, created_by	Funcionário de uma arena; <code>access_level</code> gerente ou atendente.
system_admins	FK user_id	Administradores da plataforma.
ARENA E QUADRAS		
arenas	FK owner_id	Local esportivo; endereço, contato e política de taxa. Soft delete.
courts	FK arena_id	Quadra com valor/hora. Soft delete.
court_sports	FK court_id	Esportes de cada quadra.
arena_business_hours	FK arena_id	Horários (até 2 períodos/dia).
arena_photos	FK arena_id	Galeria da arena (até 15, ordenável).
payment_methods	PK id · UK type	Formas de pagamento do sistema (pix/card/cash).
arena_payment_methods	FK arena_id, payment_method_id	Associativa N:N — formas aceitas por arena.
arena_favorites	FK client_id, arena_id	Associativa N:N — favoritas do cliente.
RESERVAS E FINANCEIRO		

TABELA	CHAVES	DESCRIÇÃO
bookings	FK court_id, client_id, created_by, payment_method_id	Reserva (site/presencial); status, taxa e cliente anulável.
payments	FK booking_id, payment_method_id, (2×) cash_register_entry	Pagamento; desconto, reembolso e vínculo ao caixa.
cash_register	FK arena_id, user_id	Caixa aberto/fechado por arena e operador.
cash_register_entries	FK cash_register_id, booking_id, created_by	Lançamentos (entrada/saída), com autor.
COMUNICAÇÃO E PLATAFORMA		
user_notifications	FK user_id, arena_id	Notificações ao usuário (por arena).
feedbacks	FK user_id	Sugestões e bugs enviados.
home_slides	PK id	Slides do carrossel da home (admin). Sem FK.

7.7 Relacionamentos

RELACIONAMENTO	CARDINALIDADE	SIGNIFICADO
<code>users</code> → <code>clients</code> / <code>owners</code> / <code>employees</code> / <code>system_admins</code>	1 : 0..1	Um usuário tem, no máximo, um perfil de cada papel (na prática, o de seu <code>type</code>).
<code>owners</code> → <code>arenas</code>	1 : N	Um proprietário possui várias arenas.
<code>arenas</code> → <code>courts</code>	1 : N	Uma arena tem várias quadras.
<code>arenas</code> → <code>arena_business_hours</code>	1 : N	Vários horários de funcionamento por arena.
<code>arenas</code> → <code>arena_photos</code>	1 : N	Galeria de fotos da arena.
<code>arenas</code> → <code>employees</code>	1 : N	Funcionários lotados na arena.
<code>arenas</code> ↔ <code>payment_methods</code>	N : N	Via <code>arena_payment_methods</code> — formas aceitas.
<code>clients</code> ↔ <code>arenas</code>	N : N	Via <code>arena_favorites</code> — arenas favoritas.
<code>courts</code> → <code>court_sports</code>	1 : N	Esportes que a quadra oferece.
<code>courts</code> → <code>bookings</code>	1 : N	Reservas de cada quadra.
<code>clients</code> → <code>bookings</code>	1 : N (opcional)	<code>client_id</code> anulável: reserva presencial não tem cliente.
<code>users</code> → <code>bookings</code>	1 : N	<code>created_by</code> : quem registrou a reserva presencial.
<code>payment_methods</code> → <code>bookings</code> / <code>payments</code>	1 : N	Forma de pagamento escolhida.
<code>bookings</code> → <code>payments</code>	1 : N	Pagamentos e reembolsos da reserva.
<code>cash_register</code> → <code>cash_register_entries</code>	1 : N	Lançamentos de cada caixa.
<code>bookings</code> → <code>cash_register_entries</code>	1 : N	Lançamentos originados de uma reserva.

RELACIONAMENTO	CARDINALIDADE	SIGNIFICADO
<code>users</code> → <code>cash_register / cash_register_entries</code>	1 : N	Operador do caixa e autor do lançamento.
<code>cash_register_entries</code> → <code>payments</code>	1 : 0..1 (×2)	Um pagamento referencia o lançamento do pagamento e o do reembolso.
<code>users / arenas</code> → <code>user_notifications</code>	1 : N	Destinatário (usuário) e remetente (arena).
<code>users</code> → <code>feedbacks</code>	1 : N	Sugestões/bugs enviados pelo usuário.

8 Arquitetura do Sistema

8.1 Arquitetura utilizada (MVC)

O sistema segue o padrão **MVC (Model–View–Controller)** do framework **Laravel**. A requisição HTTP entra por uma **rota**, passa por **middlewares** de autenticação/autorização, é tratada por um **Controller** que orchestra a lógica (apoiado por *Services*), consulta/grava dados via **Models** (Eloquent ORM sobre o MySQL) e devolve uma **View** (template Blade) renderizada em HTML. Isso separa apresentação, regras e dados, facilitando manutenção e teste.

8.2 Organização das camadas

Rotas

`routes/web.php`

Mapeiam cada URL para um controller e aplicam os grupos de middleware.



Middleware

`Http/Middleware`

`auth`, `verified`, `admin`, `pode.gerir`, `AuthenticateSession` — controlam quem acessa cada rota.



Controllers

Http/Controllers

Recebem a requisição, validam a entrada e coordenam a resposta (Cliente, Owner, Employee, Admin).



Services / Support

app/Services, app/Support

Regras reutilizáveis: `PaymentService`, `SlideImageService`, `ArenaAtual`, `Faturamento`.



Models (Eloquent)

app/Models

Entidades e relacionamentos; acesso ao MySQL, casts, escopos e soft deletes.



Views (Blade)

resources/views

HTML renderizado com componentes e partials; estilo em Bootstrap 5.

Camadas transversais: **Actions** (fluxos de autenticação do Fortify), **Notifications** (avisos ao usuário), **Traits** (código compartilhado entre controllers), **Migrations** (versão do schema) e **Lang** (traduções pt_BR).

8.3 Tecnologias utilizadas

Linguagem: `PHP 8.3`

Framework: `Laravel 12`

Autenticação: `Fortify + Jetstream`

ORM: `Eloquent`

Banco: `MySQL 8 · InnoDB`

Templates: `Blade`

UI: `Bootstrap 5 · Bootstrap Icons`

Front: `JavaScript (vanilla)`

Imagens: `GD`

Dependências: `Composer · npm`

Servidor local: `WAMP (Apache)`

Localização: `pt_BR`

9 Descrição dos Módulos

Papéis do sistema. O ArenaPlay distingue quatro papéis: **Cliente**, **Proprietário** (dono da arena), **Funcionário** (Gerente ou Atendente) e **Administrador** da plataforma. A administração da arena

(agenda, financeiro, funcionários, clientes) é do **Proprietário/Gerente**; a administração da plataforma (empresas, moderação) é do **Administrador**.

9.1 Área Pública

Sem autenticação

VISITANTE

Landing Page	Tela inicial com carrossel de destaques (gerenciado pelo admin), busca de arenas e chamada para cadastro.
Catálogo de Quadras	Listagem e pesquisa de arenas ativas; página de detalhes com a galeria de fotos, as quadras (esportes e valor/hora) e os horários.
Informações Gerais	Termos de uso, política de privacidade e a política de cancelamento da arena — visíveis antes de reservar.

9.2 Módulo Cliente

Cliente

CLIENTE

Cadastro	Criação de conta com nome, e-mail, senha e telefone; aceite dos termos.
Login	Acesso com e-mail e senha, com limite de tentativas.
Agendamento	Escolha da arena, quadra, data e horário disponível; favoritar arenas.
Pagamento	Pagamento online por PIX ou cartão (simulado), com desconto/taxa quando aplicável.
Histórico	Reservas por situação (a realizar, hoje, pendentes, histórico, canceladas, não pagas); cancelamento e reagendamento conforme a política.
Recuperação de Senha	Link enviado ao e-mail com tela de redefinição.

9.3 Módulo Funcionário

Funcionário — Gerente / Atendente FUNCIONÁRIO

Controle de Agenda	Consulta e operação das reservas da arena, incluindo o registro de reservas presenciais no balcão.
Gestão de Status	Confirmar, cancelar e reagendar reservas; operar o caixa (receber, lançar). O gerente ainda reaproveita as telas de gestão do dono; o atendente fica restrito a reservas e caixa.
Recuperação de Senha	Mesmo fluxo de link por e-mail. O funcionário já nasce com e-mail verificado (criado pela arena).

9.4 Módulo Administrador

Administração da arena PROPRIETÁRIO / GERENTE

Gestão de Agendamentos	Confirmar, cancelar, reagendar e registrar reservas (site e presencial); listas de hoje, pendentes e histórico.
Gestão Financeira	Caixa (abrir/fechar, receber com desconto, lançamentos, reembolsos e pendentes) e relatórios (balanço, lançamentos, faturamento líquido).
Gestão de Funcionários	Cadastrar, editar, ativar/desativar e excluir gerentes e atendentes.
Gestão de Clientes	Lista de clientes da arena com estatísticas; detalhes e reservas; mensagens individuais ou em massa.

Administração da plataforma

ADMINISTRADOR

Gestão de Administradores	Cadastrar administradores da plataforma; pesquisa rápida.
Gestão de Empresas e Arenas	Ver, ativar/desativar e excluir proprietários, empresas, arenas e quadras.
Usuários e Feedbacks	Bloquear/desbloquear/excluir usuários; moderar sugestões e bugs; gerenciar a aparência da tela inicial.
Recuperação de Senha	Mesmo fluxo de link por e-mail, comum a todos os papéis.

10 Manual de Instalação

Do download até o sistema rodando, na ordem correta. Vale para Windows (WAMP/XAMPP) e Linux. Este manual informa o que precisa existir na máquina — não ensina a instalar WAMP, Node ou Git (isso segue a documentação oficial de cada um).

10.1 Requisitos da máquina

COMPONENTE	VERSÃO	POR QUE É NECESSÁRIO	CONFERIR
PHP	8.3+	Exigido pelo <code>composer.json</code> . Versões menores nem instalam as dependências.	<code>php -v</code>
Composer	2.x	Instala as bibliotecas PHP (Laravel, Fortify, Sanctum).	<code>composer --version</code>
MySQL / MariaDB	MySQL 8.0+ (mín. 5.7) · MariaDB 10.2+	Uma migration cria uma coluna gerada (STORED) — recurso ausente em versões antigas, onde o <code>migrate</code> falha.	<code>mysql --version</code>
Node.js + npm	Node 18+	Compila os arquivos de estilo (assets). Sem isso, telas de estilo quebram.	<code>node -v</code>
Git	qualquer	Baixar o projeto.	<code>git --version</code>

Extensões do PHP

Costumam vir habilitadas no WAMP/XAMPP — **exceto a GD em algumas instalações Linux**.

EXTENSÃO	PARA QUE SERVE
<code>gd</code>	Obrigatória. Redimensiona e comprime as fotos. Sem ela, o envio de foto falha.
<code>pdo_mysql</code>	Conexão com o banco.
<code>mbstring</code>	Textos com acento (o sistema é em português).
<code>openssl</code>	Criptografia de senhas e sessões.
<code>fileinfo</code>	Validação real do tipo do arquivo enviado (segurança do upload).
<code>tokenizer, xml, ctype, json, curl, bcmath, zip</code>	Exigências padrão do Laravel.

Conferir/habilitar: `php -m` lista as ativas. Para habilitar uma, abra o `php.ini`, tire o `;` da frente de `extension=gd` e reinicie. Descubra qual `php.ini` está em uso com `php --ini`.

10.2 Ambiente

Windows (WAMP ou XAMPP)

Modo mais simples: você **não precisa do Apache**. Rodando `php artisan serve` na pasta do projeto, o sistema abre em `http://127.0.0.1:8000` e o WAMP/XAMPP serve apenas o **MySQL**. Neste modo, a pasta do projeto pode ficar em qualquer lugar.

Atenção: existem DOIS php.ini. O **Apache** usa um; a **linha de comando** (`php artisan serve`) usa outro. Habilitar uma extensão no arquivo errado não muda nada. Rode `php --ini` para ver qual o terminal lê — é o que importa ao usar `php artisan serve`.

MySQL: inicie o WAMP/XAMPP e verifique que o serviço do MySQL está ativo (ícone verde no WAMP). O usuário padrão costuma ser `root` **sem senha** — é o que o `.env.example` já assume.

Linux

Instale PHP 8.3 e as extensões acima (no Debian/Ubuntu cada uma é um pacote: `php8.3-mysql`, `php8.3-gd`, `php8.3-mbstring`, `php8.3-xml`, `php8.3-curl`, `php8.3-zip`, `php8.3-bcmath`), MySQL/MariaDB, Composer, Node/npm e Git.

Permissões — o erro nº 1 no Linux. O servidor precisa **escrever** em duas pastas (senão dá tela branca / erro 500):

```
sudo chown -R $USER:www-data storage bootstrap/cache
sudo chmod -R 775 storage bootstrap/cache
```

No Windows isso não é necessário.

10.3 Instalação passo a passo

Execute **nesta ordem** — cada passo depende do anterior.

1 Baixar o projeto

```
git clone https://github.com/renatorodrigues3333/trabalhoprogweb.git
cd trabalhoprogweb
```

2 Instalar dependências do PHP — cria a pasta `vendor/`.

```
composer install
```

3 Criar o arquivo de configuração (o `.env` guarda senhas e não vem no repositório).

```
copy .env.example .env      # Linux/Mac: cp .env.example .env
```

4 Gerar a chave da aplicação — sem ela, dá "419 Page Expired" nos formulários.

```
php artisan key:generate
```

5 Preencher o `.env` — confira a seção 10.4 (informe a senha do MySQL, se houver).

6 Criar o banco e as tabelas.

```
php artisan migrate
```

7 Popular os dados iniciais (obrigatório).

```
php artisan db:seed
```

8 Liberar o acesso às imagens.

```
php artisan storage:link
```

9 Compilar os estilos.

```
npm install  
npm run build
```

10 Rodar o sistema — abra `http://127.0.0.1:8000`.

```
php artisan serve
```

Não rode `composer update`. O `composer.lock` vem no repositório e trava as versões exatas testadas; `composer install` instala essas. O `update` busca versões novas e reescreve o lock — pode quebrar o projeto. Se o `install` sugerir "run composer update", **não obedeça** (ver 10.6).

Não precisa criar o banco à mão: se `arena_play` não existir, o Laravel pergunta se deve criar — responda **yes**. Em servidor use `php artisan migrate --force`.

Os dois passos mais esquecidos: `storage:link` (sem ele as fotos salvam mas dão 404) e `npm run build` (sem ele as páginas de estilo dão "Vite manifest not found"). O `db:seed` também é obrigatório: sem as formas de pagamento, não dá pra cadastrar uma arena.

10.4 Configuração do `.env`

Campos obrigatórios

CAMPO	VALOR	O QUE FAZ
APP_KEY	gerado pelo comando	Chave de criptografia. Vem vazio; o passo 4 preenche.
APP_URL	http://127.0.0.1:8000	Endereço do sistema; usado nos links dos e-mails.
DB_CONNECTION	mysql	Tipo de banco.
DB_HOST	127.0.0.1	Onde o MySQL está.
DB_PORT	3306	Porta padrão do MySQL.
DB_DATABASE	arena_play	Nome do banco (criado sozinho no passo 6).
DB_USERNAME	root	Usuário do MySQL.
DB_PASSWORD	(vazio)	No WAMP/XAMPP o padrão é sem senha.

Campos que já vêm certos (não mexa sem motivo)

CAMPO	VALOR	POR QUÊ
APP_ENV	local	Modo de desenvolvimento.
APP_DEBUG	true	Mostra os erros na tela. Em produção → <code>false</code> .
MAIL_MAILER	log	E-mails não são enviados : vão para <code>storage/logs/laravel.log</code> .
EMAIL_VERIFICATION_ENABLED	false	Não exige confirmar o e-mail. Só ligue depois de configurar o SMTP.
QUEUE_CONNECTION	sync	Envia os e-mails na hora (sem processo em segundo plano).
SESSION_DRIVER / CACHE_STORE	file	Sessão e cache em arquivo — não exigem tabela.

Opcional — enviar e-mails de verdade: troque `MAIL_MAILER=smtp` e preencha

`MAIL_HOST/PORT/USERNAME/PASSWORD/SCHEME` (ex.: Gmail exige uma "senha de app", não a senha normal). Só ligue `EMAIL_VERIFICATION_ENABLED=true` **depois** que o envio funcionar — senão trava os cadastros novos. Após mexer no `.env` : `php artisan config:clear` .

Opcional — fotos maiores: o padrão do PHP aceita 2 MB (o sistema já encolhe a foto no navegador, então não é obrigatório). Para folga, no `php.ini` : `upload_max_filesize=8M` e `post_max_size=12M` (o post precisa ser maior), e reinicie. Sobre sessão/cache/fila em produção, ver a **seção 12**.

10.5 Verificação final

COMANDO / CHECAGEM	RESPOSTA ESPERADA
<code>php artisan migrate:status</code>	Lista as migrations, todas como Ran .
<code>php artisan route:list</code>	Lista as rotas sem erro (o sistema sobe).
Existe a pasta <code>public/storage</code> ?	Sim (passo 8).
Existe a pasta <code>public/build</code> ?	Sim (passo 9).
Abrir <code>/</code> , <code>/login</code> , <code>/forgot-password</code>	Abrem sem erro.

10.6 Problemas comuns

MENSAGEM / SINTOMA	CAUSA	SOLUÇÃO
<i>Vite manifest not found</i>	Estilos não compilados.	<code>npm install</code> + <code>npm run build</code> (passo 9).
<i>[1049] Unknown database</i>	O banco não existe.	<code>php artisan migrate</code> e responda yes ; confira <code>DB_DATABASE</code> .
<i>[1045] Access denied</i>	Usuário/senha do MySQL errados.	Ajuste <code>DB_USERNAME/DB_PASSWORD</code> + <code>config:clear</code> .
<i>[2002] Connection refused</i>	MySQL não está rodando.	Inicie o serviço no WAMP/XAMPP.
419 Page Expired ao enviar formulário	<code>APP_KEY</code> vazio.	<code>php artisan key:generate</code> (passo 4).
Fotos dão 404	Falta o atalho das imagens.	<code>php artisan storage:link</code> (passo 8).
<i>undefined function imagecreatefromjpeg()</i>	Extensão GD desabilitada.	Habilite <code>extension=gd</code> e reinicie.
Nenhuma forma de pagamento ao cadastrar arena	<code>db:seed</code> não foi executado.	<code>php artisan db:seed</code> (passo 7).
O e-mail não chega	<code>MAIL_MAILER=log</code> : é gravado, não enviado.	Veja <code>storage/logs/laravel.log</code> ou configure o SMTP.
Mudei o <code>.env</code> e nada mudou	Config em cache.	<code>php artisan config:clear</code>
Erros estranhos após instalar (classe/método inexistente)	Rodaram <code>composer update</code> .	<code>git checkout composer.lock</code> + <code>composer install</code> .

A causa raiz da maioria dos erros graves é o `composer update`. Ele derruba um pacote de que a configuração depende e **nenhum comando artisan funciona mais**. A saída é sempre voltar ao `composer.lock` do repositório (`git checkout composer.lock` + `composer install`), nunca gerar um novo por conta própria.

10.7 Acesso inicial

O `db:seed` cria um administrador: `adminteste@gmail.com` · senha `12345678` · painel em `/admin`. **Nunca leve essa senha para produção** — troque-a no primeiro acesso. Os outros perfis se criam pelo site: **Cliente** em `/register`, **Proprietário** em `/registerArenaOwners` (4 etapas), e **Funcionário** pelo painel do proprietário. Para um cenário completo de teste, ver a **seção 11**.

10.8 Sobre os arquivos e o banco

ITEM	ONDE FICA	VAI PARA O GIT?
Imagem padrão da home	<code>public/img/home-default.jpg</code>	Sim — garante que a home nunca quebre
Fotos enviadas	<code>storage/app/public</code>	Não — são por máquina
Configurações e senhas	<code>.env</code>	Não — criado em cada máquina
Bibliotecas PHP	<code>vendor/</code>	Não — vem do <code>composer install</code>
Estilos compilados	<code>public/build</code>	Não — vem do <code>npm run build</code>

Backup: o *dump* do banco **não basta** — salve também a pasta `storage/app/public`, onde ficam as imagens enviadas. Numa máquina nova, se as imagens não vierem, o sistema mostra a imagem padrão no lugar (sem quebrar) e basta reenviá-las.

11 Manual de Teste Local

Como acessar

- 1 Popule o **cenário de teste** (usuários, arena, quadras e reservas de exemplo):

```
php artisan db:seed --class=DadosDeTesteSeeder
```

Idempotente — pode rodar quantas vezes quiser; recria o cenário sem duplicar.

- 2 Inicie o servidor: `php artisan serve`
- 3 Abra **http://127.0.0.1:8000** e clique em **Entrar**.

Usuários de teste

Todos com a senha **12345678**.

PAPEL	E-MAIL (LOGIN)	O QUE TESTAR
Administrador	adminteste@gmail.com	Painel da plataforma (<code>/admin</code>): usuários, empresas, arenas, feedbacks, aparência.
Proprietário	dono@teste.com	Gestão da "Arena Teste": quadras, fotos, horários, taxa, caixa, clientes e funcionários.
Funcionário — Gerente	gerente@teste.com	Mesmas telas de gestão do dono.
Funcionário — Atendente	atendente@teste.com	Painel próprio: reserva presencial e caixa (sem gestão).
Cliente	cliente@teste.com	Reservar, pagar, cancelar e acompanhar o histórico.
Cliente 2	cliente2@teste.com	Segundo cliente, com reservas em estados variados.

Fluxo para testar

O seeder já deixa a **"Arena Teste"** com 3 quadras e 6 reservas em estados variados (a confirmar, confirmada, paga, cancelada, realizada e presencial), então todas as telas já aparecem povoadas.

Como cliente CLIENTE@TESTE.COM

Buscar e reservar	Na home, pesquise "Arena Teste", veja a galeria e as quadras, e faça uma nova reserva escolhendo data e horário.
Pagar	Em "Minhas reservas", pague uma reserva confirmada por PIX/cartão (simulado).
Cancelar	Cancele uma reserva e observe a taxa/reembolso conforme a política da arena.
Recuperar senha	Saia, clique em "Esqueceu a senha?" e confira o link gerado em <code>storage/logs/laravel.log</code> (o e-mail vai para o log no ambiente local).

Como proprietário / gerente DONO@TESTE.COM · GERENTE@TESTE.COM

Arena e quadras	Edite a arena, adicione/reordene fotos, ajuste horários e a taxa de cancelamento; crie/edite quadras.
Reservas	Confirme, cancele ou reagende as reservas do cenário.
Caixa	O caixa já está aberto: receba uma reserva (com ou sem desconto), faça um lançamento avulso e veja os relatórios.
Clientes e equipe	Veja a lista de clientes com estatísticas, envie uma mensagem, e gerencie os funcionários.

Como atendente ATENDENTE@TESTE.COM

Reserva presencial	Registre uma reserva de balcão para um cliente sem cadastro.
Caixa	Receba pagamentos e faça lançamentos; note que as telas de gestão (lucro, funcionários) ficam bloqueadas.

Como administrador ADMINTESTE@GMAIL.COM

Moderação Em `/admin`, veja indicadores, gerencie usuários/empresas/arenas, trate feedbacks e edite a aparência da tela inicial.

12 Melhorias Futuras

Limitações conscientes do estágio atual e as evoluções previstas. As configurações de produção correspondentes estão descritas na seção 10.

Pagamento online real DESTAQUE

Hoje o pagamento online é **simulado**. A evolução natural é integrar um **gateway real** (ex.: Mercado Pago, Stripe ou PIX via API de um PSP/banco), tratando o **webhook de confirmação**, a **conciliação no caixa** e o **estorno real**.

O sistema já está **preparado** para isso: a tabela `payments` (com `origin`, `status` e campos de reembolso), o serviço `PaymentService` e as formas de pagamento já modeladas permitem **trocar apenas o passo simulado** pelas chamadas ao gateway — sem redesenhar o modelo de dados.

Outras evoluções previstas

MELHORIA	SITUAÇÃO ATUAL	EVOLUÇÃO PREVISTA
Envio de e-mail	Gravado no log (<code>MAIL_MAILER=log</code>) — não é entregue.	Configurar SMTP e ligar a verificação de e-mail (já implementada por link).
Processamento em fila	<code>QUEUE_CONNECTION=sync</code> — tudo na hora.	Fila em banco + <code>queue:work</code> : e-mails e tarefas em segundo plano, sem travar a tela.
Segurança em produção	Regras mínimas; ambiente de desenvolvimento.	HTTPS , <code>APP_DEBUG=false</code> e política de senha forte (rejeitar senhas vazadas).
Escalabilidade	Listas paginadas; 2 custos $O(n)$ mapeados.	Numeração via coluna e busca FULLTEXT para dezenas de milhares de registros.
